

UNIVERSIDADE DO VALE DO RIO DOS SINOS — UNISINOS
PÓS-GRADUAÇÃO — CULTURA E PRÁTICAS DEVOPS E DEVSECOPS

ESTEIRA DEVSECOPS APLICADA AO PROJETO INNER VIEW

Diagnóstico, pipeline de CI/CD, controles de segurança e métricas de acompanhamento

Integrantes do grupo

Fabricao Varisco

Gustavo Potrich

Lucas Krug

Matheus Thiesen

Repositório: github.com/fabvarisco/ts-inner-view

Módulo 4 — Atividade avaliativa · Setembro de 2026

1 CONTEXTUALIZAÇÃO E DIAGNÓSTICO

1.1 O projeto e a forma atual de trabalho

O **Inner View**, trabalho final da pós-graduação do grupo, é uma plataforma de *tours virtuais imobiliários*: o corretor cadastra o imóvel, envia as fotos panorâmicas de cada ambiente, liga um ambiente ao outro por pontos de navegação e publica um tour que o cliente percorre à distância. São duas aplicações no mesmo repositório: a API `server-api` (NestJS, TypeScript, Prisma, PostgreSQL, autenticação por JWT, 26 rotas e 12 modelos de dados) e o aplicativo `inner-view-client` (Angular, Ionic, Capacitor para Android, Three.js no visualizador 360°). O ambiente local já está descrito em código, com `Dockerfile` e `docker-compose.yml`.

A forma de trabalho aparece no histórico do Git: **19 commits** entre 09/06/2026 e 26/09/2026, **uma única branch** (`main`) e **nenhum pull request**. Todos comitam direto na `main`, sem revisão. O diretório `.github/` não existia, ou seja, **não havia nenhuma automação**. Cada integrante sobe o banco, roda o projeto e testa na própria máquina — o que funciona, funciona só ali.

1.2 Diagnóstico do estado atual

Para o diagnóstico não ficar no "achismo", rodamos no repositório as mesmas verificações que a esteira faria. As saídas estão no Anexo F e o resumo no Quadro 1.

Dimensão	Estado atual observado (com evidência)
Integração de código	Commits diretos em <code>main</code> , sem revisão, sem verificação e sem proteção de branch — 19 commits, 0 merges, 0 PRs
Testes	Os testes de aceitação do módulo de tours virtuais, escritos em TDD, são 52 e passam todos ; mas a cobertura da API inteira é de 28% das linhas (autenticação, imóveis, usuários e panoramas não têm teste) e não existem testes de integração. No aplicativo, dos 5 testes existentes, 3 estão quebrados (erro NG0201) — vieram do esqueleto do Angular, nunca foram atualizados e estão assim na <code>main</code> há vários commits sem ninguém perceber
Qualidade	O comando <code>yarn lint</code> do projeto corrige em vez de verificar, então nunca reprova. Rodando só como verificação, aponta 89 erros na API (88 deles automáticos) e 1 no aplicativo
Build e deploy	Os builds funcionam, mas a imagem é gerada na máquina de quem desenvolve e não é publicada em lugar nenhum. Existe um projeto criado na Vercel para o aplicativo, e nenhum deploy sai dele — não há esteira que o acione. Na prática, não existe deploy : em 3,5 meses, o sistema nunca saiu do <code>localhost</code> . O endereço da API no build de produção é <code>https://api.seudominio.com</code> , um domínio que não existe

Segurança do código e segredos	Nenhuma verificação automática. Lendo o código de autenticação, achamos um segredo reserva escrito no próprio código: <code>process.env.JWT_ACCESS_SECRET</code> ?? <code>'access-secret'</code> — se a variável de ambiente faltar, a API assina os tokens com um segredo público. Do lado positivo, o <code>.env</code> está no <code>.gitignore</code> e nenhum segredo real foi versionado; mas o <code>.env.example</code> traz exemplos fracos (<code>JWT_ACCESS_SECRET=aaaa</code>) que acabam virando valor real, e ninguém varre o histórico atrás de segredos
Dependências	API: 136 alertas de risco alto em 827 bibliotecas. Aplicativo: 1 crítico e 35 altos em 1.608 bibliotecas. Não existe inventário de dependências (SBOM) nem atualização automática
Configuração da API	A API sobe sem CORS definido, sem limite de requisições e sem cabeçalhos de segurança; a documentação Swagger fica aberta; o limite de upload é de 50 MB por requisição (panoramas em base64); e o contêiner roda como <code>root</code>
Monitoramento	Não existe: sem rota de saúde, sem log organizado e sem alerta. Sem isso, não dá para medir quanto tempo se leva para detectar ou corrigir uma falha

Quadro 1 — Diagnóstico do estado atual. Fonte: elaborado pelos autores; saídas completas no Anexo F.

Em resumo: o projeto tem partes técnicas boas e nenhuma esteira que ligue uma coisa na outra. O achado que melhor resume tudo não é a lista de vulnerabilidades, e sim os **três testes quebrados na branch principal sem ninguém saber** — o problema não é falta de cuidado técnico, é falta de um mecanismo que avise o grupo.

2 O DESENHO DO PIPELINE DE CI/CD

2.1 Da cultura ao desenho: Três Caminhos e CALMS

No Inner View não existe o muro entre desenvolvimento e operação de *O Projeto Fênix*: existe **ausência de operação**, porque não há o que operar fora do `localhost`. Os sintomas, porém, são os mesmos dos Três Caminhos: falta **fluxo** (o trabalho não chega ao usuário), falta **feedback** (os três testes quebrados provam) e falta **aprendizado contínuo** (nada é medido). A esteira responde aos três: entregas pequenas e um único artefato promovido criam o fluxo; lint, testes e verificações de segurança respondem em minutos no próprio *pull request*; e as métricas da seção 4 fecham o aprendizado.

Usamos o modelo **CALMS** como lista de conferência, para a proposta não virar só automação. **Cultura**: a `main` passa a ser responsabilidade de todos — esteira vermelha vira prioridade do grupo, e falha vira análise sem procurar culpado. **Automação**: quatro workflows cobrindo lint, build, teste, segurança e deploy, onde hoje não há nada. **Lean**: branches de até dois dias e pull requests pequenos, no lugar de entregas grandes que nunca chegam ao usuário. **Medição**: as nove métricas da seção 4, coletadas pela própria esteira. **Compartilhamento**: revisão por outro integrante e rodízio de quem publica a release, para o conhecimento não ficar com uma pessoa só.

2.2 Branches, ambientes e critérios de promoção

A estratégia escolhida é **trunk-based development**: a `main` fica sempre pronta para entregar, e cada tarefa vive em uma branch curta (`feat/`, `fix/`) de no máximo dois dias, que volta para a `main` por *pull request*. Descartamos o GitFlow porque, para quatro pessoas e entregas pequenas, as branches extras só atrasariam o feedback. A `main` fica protegida no GitHub: sem pull request aprovado por um colega e sem a esteira verde, não entra nada.

Ambiente	O que dispara	Artefato	Finalidade
Local	<code>docker compose up</code> <code>eng serve</code>	Build na máquina	Desenvolvimento, com dados de <i>seed</i>
Preview	Abertura ou atualização de <i>pull request</i>	<i>Deployment</i> efêmero na Vercel, uma URL por PR	Revisar a mudança rodando, e não só o <i>diff</i>
Homologação	Merge em <code>main</code> (automático)	<i>Deployment</i> imutável na Vercel, indexado pelo SHA do commit	Smoke test e alvo do DAST; ainda sem domínio público
Produção	<i>Tag</i> <code>v*</code> + aprovação manual	O mesmo deployment validado em homologação	Domínio público, variáveis próprias, acesso de usuário

Quadro 2 — Ambientes e critérios de promoção. Fonte: elaborado pelos autores.

A API não aparece no quadro como ambiente porque ainda não tem host contratado. O que a esteira já faz por ela é **empacotar**: cada merge gera a imagem, ela passa pelo scan e é publicada no GHCR identificada pelo SHA do commit. Quando houver host, o artefato já existe, versionado e com inventário — falta apenas o passo que o coloca no ar.

Duas decisões sustentam o desenho. A primeira: o artefato é **construído uma vez só**, no merge em `main`, e daí em diante apenas promovido — o que chega à produção é exatamente o que foi testado. Na Vercel isso tem tradução direta: o merge roda `vercel deploy --prebuilt --prod --skip-domain`, que cria um *deployment* de produção com URL própria mas **sem apontar o domínio** para ele; a promoção, na *tag*, é um `vercel promote` que apenas move o domínio para esse mesmo *deployment*. A segunda: a *tag* no Git é o **gatilho da produção**, de modo que o grupo decide quando liberar sem perder a automação do caminho. Como todo *deployment* anterior continua existindo e imutável, voltar atrás é mover o ponteiro de volta.

2.3 Do commit ao deploy

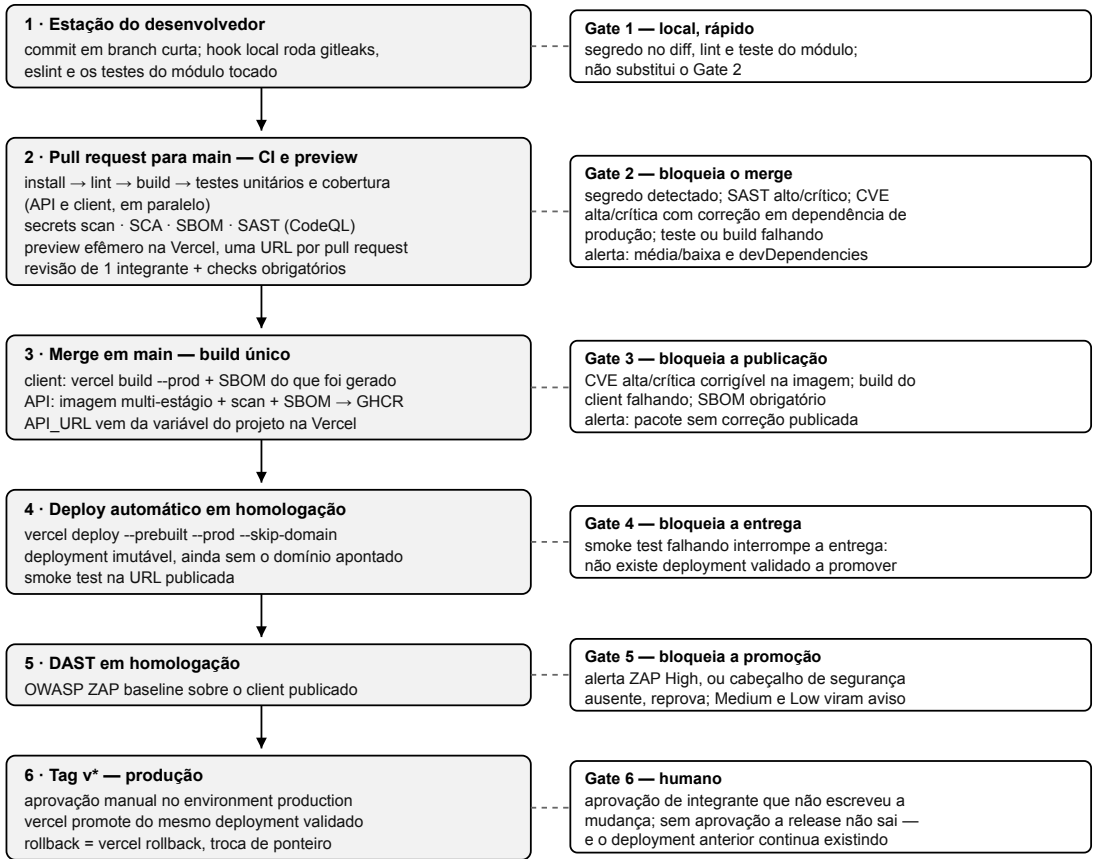


Figura 1 — Esteira do commit ao deploy e os seis gates. Fonte: elaborada pelos autores.

2.4 Ferramentas escolhidas e justificativa

Ferramenta	Papel	Por que esta
GitHub Actions	Executa a esteira	O projeto já está no GitHub: não exige servidor nem credencial nova, e já traz proteção de branch, aprovação manual de deploy e painel de segurança. Jenkins exigiria manter um servidor, sem ganho para quatro pessoas
Gitleaks	Procura segredos	Varre o histórico inteiro, e não só a última alteração — um segredo apagado hoje continua visível nos commits antigos. É gratuito e roda também na máquina do desenvolvedor
GitHub CodeQL	SAST: analisa o código	Já integrado ao GitHub e sem custo. Entende TypeScript de verdade (segue o caminho do dado, não procura texto) e aponta injeção de código, criptografia fraca e segredo escrito no código — o mesmo tipo de problema que achamos no diagnóstico. O resultado aparece como comentário no pull request
Trivy	SCA: analisa dependências e a imagem	Uma ferramenta só cobre as bibliotecas do projeto e os pacotes de dentro do contêiner. A opção <code>--ignore-unfixed</code> é o que torna o bloqueio possível: das 136 vulnerabilidades altas de hoje, só barram as que já têm correção disponível
Syft	SBOM: lista de ingredientes	Gera a lista de tudo que entrou em cada versão, no formato CycloneDX. Quando sair uma vulnerabilidade nova, responde na hora "essa versão usa essa biblioteca?"

OWASP ZAP	DAST: testa a aplicação rodando	Gratuito e em modo passivo, seguro para rodar em homologação a cada versão. Enxerga o que a análise de código não vê: cabeçalhos de segurança ausentes, CORS mal configurado e rotas expostas. Como cada <i>deployment</i> da Vercel tem URL própria, o DAST roda sempre contra a versão exata que se pretende promover
Dependabot	Atualiza dependências	Abre pull requests semanais pequenos com as atualizações, cada um passando pela esteira. Sem ele, o SCA só acusa e ninguém corrige
Vercel	Hospeda e publica o aplicativo	O grupo já tinha o projeto criado lá e o plano gratuito atende: some a necessidade de contratar e manter servidor, que era o que travava o deploy. Cada publicação é um <i>deployment</i> imutável com URL própria — o que dá preview por <i>pull request</i> , alvo estável para o DAST, promoção sem novo build e <i>rollback</i> em segundos. A integração Git automática foi desligada em <code>vercel.json</code> para que quem publique seja a esteira, depois dos gates, e não o <i>push</i>
GHCR	Guarda a imagem da API	Registry do próprio GitHub: mesma conta, cada imagem identificada pelo commit e guardada. É onde a API fica versionada enquanto não há host — e o que tornará o <i>rollback</i> dela imediato quando houver
Jest e Karma	Testes	Já usados no projeto, inclusive nos 52 testes em TDD do módulo anterior. A esteira não troca de ferramenta: apenas passa a rodá-las de forma obrigatória

Quadro 3 — Ferramentas e justificativa da escolha. Fonte: elaborado pelos autores.

Essa esteira já está no repositório: quatro workflows (Anexos A a D), mais Dependabot e a configuração de entrega na Vercel (Anexo E). Diferente de um desenho de esteira só no papel, **não há passo de publicação em aberto para o aplicativo** — o destino existe, a esteira executa o deploy e o rollback é um comando. O que continua pendente é o host da API; até lá, a esteira entrega a imagem pronta e verificada no registry.

2.5 Implantação e consistência entre ambientes

A unidade de entrega do aplicativo é o ***deployment* imutável** da Vercel. Uma vez criado, ele não muda: recebe uma URL própria, derivada do build, e continua existindo mesmo depois de substituído. "Publicar" e "voltar atrás" deixam de ser operações de infraestrutura e viram a mesma operação — apontar o domínio para um *deployment* que já existe. É o que o `vercel promote` faz na tag e o `vercel rollback` faz quando o *health check* de produção reprova, sem novo build e sem janela de indisponibilidade. O efeito prático é o mesmo que se buscaria com **blue-green**, só que sem manter duas infraestruturas de pé: a versão nova é validada em uma URL real antes de receber o domínio.

Para a API, a unidade continua sendo o **contêiner**: o mesmo `Dockerfile` multi-estágio gera a imagem que será usada em qualquer ambiente, o que elimina o clássico "funcionava em homologação". Como as *migrations* rodam ao subir o contêiner e, em uma troca de versão, duas convivem por instantes, toda migration precisa ser compatível com a versão anterior — adicionar coluna em vez de renomear, e remover só na entrega seguinte. Essa regra vale desde já, mesmo

antes de existir *host*, porque é uma disciplina de como escrever a *migration* e não de como fazer o *deploy*.

Duas limitações deste desenho ficam registradas, porque ignorá-las seria vender uma esteira melhor do que ela é:

- **Homologação e produção compartilham a configuração.** Construir uma vez só e promover o mesmo artefato tem um custo: em uma aplicação Angular a configuração entra no *bundle* durante o *build*, então o *deployment* validado em homologação já carrega as variáveis de produção. Hoje isso é inofensivo, porque a API não está no ar; quando estiver, ou se abre mão de promover o mesmo artefato — construindo duas vezes —, ou se passa a ler a configuração em tempo de execução, carregando um `config.json` servido pelo ambiente antes de a aplicação iniciar. O grupo prefere o segundo caminho, e ele está na fase 3 do plano.
- **O *canary* fica para depois.** Liberar para uma fração dos usuários só faz sentido com usuários reais e com monitoramento que mostre a diferença entre as duas fatias. Sem rota de saúde e sem log organizado, um *canary* daria a sensação de cuidado sem a informação que o justifica.

3 CONTROLES DE SEGURANÇA INTEGRADOS À ESTEIRA

A regra que organiza os gates é simples: **bloqueia o que dá para resolver agora, avisa o resto**. Um gate que reprova por algo que o grupo não tem como corrigir ensina o time a ignorar a esteira — e esteira ignorada é pior do que nenhuma, porque dá falsa sensação de segurança.

Controle	Ferramenta	Gate	Bloqueia quando	Apenas alerta quando
Secrets scanning	Gitleaks	1 (local) e 2 (PR, histórico completo)	Qualquer segredo com padrão reconhecido é detectado — é o único controle bloqueante desde o primeiro dia	Nada. Falso positivo vira <i>allowlist</i> revisada em PR, não afrouxamento do gate
SAST	CodeQL security-extended	2 (PR) e agenda semanal	Achado alto ou crítico introduzido pelo <i>pull request</i>	Média e baixa; achados pré-existentes, que entram na fila de correção
SCA	Trivy fs	2 (PR) e agenda semanal	CVE alta ou crítica com correção publicada em dependência de produção	Média e baixa; <code>devDependencies</code> ; CVE sem correção disponível, que gera <i>issue</i> com prazo
SBOM	Syft (CycloneDX)	2 (código) e 3 (imagem da API e build do aplicativo)	Falha na geração — nenhum artefato é publicado sem inventário	Diferenças de inventário entre releases, discutidas na revisão

Scan de imagem	Trivy image	3 (antes do push)	CVE alta ou crítica corrigível no sistema operacional ou nas bibliotecas da imagem	Pacote sem correção <i>upstream</i> : registra-se exceção com prazo de revisão
Configuração de ambiente	set-api-url.mjs	3 (build)	API_URL não está definida ou não é uma URL válida — o build falha em vez de publicar um aplicativo que aponta para lugar nenhum	—
Smoke test	curl sobre a URL do <i>deployment</i>	4 (homologação) e 6 (produção)	A URL publicada não devolve 200 com a aplicação dentro. Sem smoke verde não há <i>deployment</i> validado para promover; em produção, dispara o <i>rollback</i>	Latência acima do esperado no smoke test
DAST	OWASP ZAP <i>baseline</i>	5 (homologação publicada)	Alerta High do ZAP, ou ausência de HSTS, CSP, X-Frame-Options e X-Content-Type-Options — reprova a promoção para produção	Medium e Low, marcados como WARN em <code>.zap/rules.tsv</code>
Aprovação humana	<i>Environment</i> protegido	6 (produção)	Falta de aprovação de integrante que não escreveu a mudança	—

Quadro 4 — Controles de segurança, gates e política de bloqueio. Fonte: elaborado pelos autores.

Quatro escolhas merecem registro, porque é nelas que a esteira encosta na realidade do projeto:

- **O problema antigo avisa; o problema novo bloqueia.** As 89 falhas de lint, os 3 testes quebrados e as vulnerabilidades que já existiam entram como aviso, marcadas com `TODO` nos workflows. Se entrassem como bloqueio, a `main` nasceria vermelha e o grupo se acostumaria a ignorar o alerta. Cada `TODO` tem prazo na fase 1 do plano da seção 5.
- **Segredo é a exceção.** Segredo vazado não tem conserto depois: precisa ser trocado. Por isso esse gate bloqueia desde o primeiro dia, e o segredo reserva `?? 'access-secret'` será trocado por um erro na inicialização — sem a variável, a API não sobe, em vez de subir insegura.
- **Cada ambiente guarda os próprios segredos.** Os segredos de produção ficam nos *environments* do GitHub e só são acessíveis ao deploy aprovado; a configuração da aplicação (o `API_URL`) fica nas variáveis do projeto na Vercel, separada por *environment*; e o `.env.example` passa a ter marcadores em vez de valores que parecem reais. A esteira precisa de três segredos para publicar — `VERCEL_TOKEN`, `VERCEL_ORG_ID` e `VERCEL_PROJECT_ID` — e de nenhuma credencial de servidor, porque não há servidor.
- **Desligamos o deploy automático da Vercel.** Por padrão a Vercel publica sozinha a cada *push*, o que colocaria o deploy *ao lado* dos gates em vez de *depois* deles: o código iria ao ar enquanto o SAST ainda roda. Em

`vercel.json` a integração Git foi desativada (`"github": { "enabled": false }`) e quem publica é o workflow, pela CLI. Perde-se um pouco de comodidade e ganha-se a única coisa que faz um gate ser gate: a capacidade de impedir.

4 MÉTRICAS DE ACOMPANHAMENTO

As quatro métricas DORA, mais MTDD e MTTR, medem a esteira; as três últimas medem a dívida que ela precisa reduzir. Todas saem de dados que a própria esteira gera, para ninguém precisar medir na mão — medição manual é medição abandonada.

Métrica	Como medir	Situação atual	Meta em 90 dias
Frequência de deploy	Execuções bem-sucedidas de <code>cd.yml</code> por ambiente	Zero. Em 3,5 meses e 19 commits, nenhum deploy: há projeto criado na Vercel, mas nada sai dele; o software só roda em <code>localhost</code>	Homologação a cada merge (≥ 3 /semana); produção ≥ 1 /semana
Lead time para mudanças	Do <i>timestamp</i> do commit ao fim do deploy que o contém	Não mensurável — não há destino; hoje depende de cada pessoa rodar o projeto	Commit → homologação < 20 min (CI + build + deploy); commit → produção < 2 dias
Taxa de falha de mudanças	Releases com <i>hotfix</i> ou <i>rollback</i> ÷ total de releases	Não medida. Indício indireto: 3 de 5 testes do front quebrados em <code>main</code> sem detecção	< 15%
MTDD — falha funcional	Do push ao primeiro <i>check</i> vermelho	Indefinido: depende de alguém rodar o teste localmente	< 10 min (duração do CI)
MTDD — vulnerabilidade	Da publicação da CVE ao alerta na aba Security	Indefinido: sem varredura, nenhuma CVE é detectada	< 7 dias com a agenda semanal; < 24 h ao passar para diária
MTTR	Do alerta ao deploy da correção	Não aplicável — sem produção, não há restauração a medir	< 4 h; <i>rollback</i> na Vercel < 2 min, por ser troca de ponteiro e não novo build
Cobertura de testes	<code>test:cov</code> publicado como artefato do CI	API: 28,0% statements e 19,1% branches , com os 52 testes concentrados em tours virtuais. Client: efetivamente nula, com 3 specs falhando	API $\geq 50\%$ com gate bloqueante; client com os 3 specs corrigidos e fluxos principais cobertos
Vulnerabilidades altas e críticas corrigíveis	Relatório do Trivy em cada execução	Client: 1 crítica e 35 altas . API: 136 alertas altos . Zero SBOM gerado	Zero em produção; SBOM em 100% dos builds
Mudanças com revisão	Commits em <code>main</code> via PR aprovado ÷ total	0% — 19 commits diretos, nenhum merge	100%

Quadro 5 — Métricas: situação atual e meta. Fonte: elaborado pelos autores.

Duas metas dependem do que falta hoje: sem rota de saúde e sem log organizado, MTDD e MTTR de produção continuam sendo chute. Por isso o monitoramento mínimo está na fase 3 do plano.

5 PLANO DE ADOÇÃO E CONSIDERAÇÕES FINAIS

A adoção é dividida em três fases, para o grupo não tentar ligar tudo de uma vez. **Fase 1 (semanas 1 e 2):** ligar o CI e o secrets scanning, proteger a `main` exigindo pull request revisado, e limpar a dívida que hoje só gera aviso — corrigir os 3 testes do aplicativo, as 89 falhas de lint e o segredo reserva no código. **Fase 2 (semanas 3 e 4):** cadastrar os três segredos da Vercel, tornar SAST, SCA e SBOM obrigatórios, ligar o Dependabot, exigir 50% de cobertura e fazer o primeiro deploy automático em homologação — o primeiro da história do projeto. **Fase 3 (semanas 5 a 8):** DAST em homologação, produção por *tag* com aprovação e *rollback* por promoção, leitura da configuração em tempo de execução para separar de fato homologação de produção, ajustes de segurança da API (CORS, cabeçalhos, limite de requisições, contêiner sem `root`, Swagger fechado em produção), definição do host da API e o monitoramento mínimo que torna MTDD e MTTR mensuráveis.

O trabalho mostrou que aplicar DevSecOps a um projeto real depende menos de escolher ferramentas — as que usamos são gratuitas e padrão de mercado — e mais de aceitar um diagnóstico desconfortável. O Inner View tem testes em TDD, arquitetura organizada e Docker bem feito e, ao mesmo tempo, tinha três testes quebrados na branch principal, um segredo escrito no código de autenticação e nenhum ambiente além da máquina de cada integrante. É o que acontece quando a qualidade depende da disciplina de cada um em vez de estar automatizada na esteira. A proposta ataca essa lacuna: devolve resposta rápida ao grupo, coloca a segurança no mesmo pull request que o teste em vez de num portão final, separa o que bloqueia do que só avisa e torna o caminho até a produção repetível. A escolha da Vercel para hospedar o aplicativo fechou a última pendência do desenho anterior: o destino existe, o deploy é executado pela esteira e o rollback é um comando, de modo que o único passo ainda no papel é o host da API. Todo o restante está versionado no repositório e reproduzido nos anexos.

REFERÊNCIAS

FORSGREN, N.; HUMBLE, J.; KIM, G. *Accelerate: the science of lean software and DevOps*. Portland: IT Revolution Press, 2018.

HUMBLE, J.; FARLEY, D. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Boston: Addison-Wesley, 2010.

KIM, G.; BEHR, K.; SPAFFORD, G. *O Projeto Fênix: um romance sobre TI, DevOps e sobre ajudar o seu negócio a vencer*. Rio de Janeiro: Alta Books, 2018.

KIM, G. et al. *The DevOps handbook*. 2. ed. Portland: IT Revolution Press, 2021.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. *SP 800-218: Secure Software Development Framework (SSDF)*. Gaithersburg: NIST, 2022.

OWASP FOUNDATION. *OWASP Top 10: 2021*. Disponível em: owasp.org/Top10. Acesso em: 26 set. 2026.

OWASP FOUNDATION. *OWASP DevSecOps Guideline*. Disponível em: owasp.org/www-project-devsecops-guideline. Acesso em: 26 set. 2026.

ANEXO A — .GITHUB/WORKFLOWS/CI.YML

Integração contínua das duas aplicações. Os `continue-on-error` marcam a dívida herdada descrita em 1.2 e têm prazo de remoção na fase 1 do plano da seção 5.

```

# Esteira de integração contínua do Inner View.
# Gate 2 do documento da disciplina: roda em todo pull request para main e em todo
push em main.
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  workflow_dispatch:

# Um PR que recebe vários pushes cancela as execuções anteriores.
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  api:
    name: API · lint, build e testes
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: server-api
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: yarn
          cache-dependency-path: server-api/yarn.lock

      - name: Instalar dependências
        run: yarn install --frozen-lockfile

      # O script `yarn lint` do projeto roda com --fix e por isso nunca falha.
      # Na esteira o lint precisa ser verificação, não correção.
      - name: Lint (verificação, sem --fix)
        # TODO fase 1: remover o continue-on-error depois de rodar `yarn lint` uma vez
        # e commitar a formatação (hoje há 89 erros, 88 deles auto-corrigíveis).
        continue-on-error: true
        run: npx eslint "{src,apps,libs,test}/**/*.ts" --max-warnings=0

      - name: Build
        run: yarn build

      - name: Testes unitários com cobertura
        run: yarn test:cov

      - name: Verificar limite mínimo de cobertura
        # TODO fase 1: elevar o limite para 50% e tornar bloqueante (hoje: 28% de
statements).
        continue-on-error: true
        run: npx jest --coverage --coverageThreshold='{ "global":
{"statements":50,"branches":40,"lines":50}}'

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: coverage-api
          path: server-api/coverage

```

```
        retention-days: 14

client:
  name: Client · lint, build e testes
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: inner-view-client
  env:
    CHROME_BIN: /usr/bin/google-chrome
  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-node@v4
      with:
        node-version: 22
        cache: npm
        cache-dependency-path: inner-view-client/package-lock.json

    # O projeto versiona package-lock.json e yarn.lock. A esteira padroniza npm;
    # o yarn.lock deve ser removido do repositório para não divergirem.
    - name: Instalar dependências
      run: npm ci

    - name: Lint
      # TODO fase 1: remover o continue-on-error (hoje há 1 erro: prefer-inject em
language.service.ts).
      continue-on-error: true
      run: npx ng lint

    - name: Build de produção
      run: npx ng build --configuration production

    - name: Testes unitários
      # TODO fase 1: remover o continue-on-error depois de corrigir os 3 specs que
falham
      # por falta do provider HttpClient (app, home e inner-view-page).
      continue-on-error: true
      run: npx ng test --watch=false --browsers=ChromeHeadless --code-coverage

    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: coverage-client
        path: inner-view-client/coverage
        retention-days: 14
```

ANEXO B — .GITHUB/WORKFLOWS/SECURITY.YML

Secrets scanning, SCA e SBOM. Roda em todo pull request e também às segundas-feiras, para detectar CVEs publicadas sem que o código tenha mudado.

```

# Controles de DevSecOps que atuam no Gate 2 (pull request) e semanalmente sobre o
código parado.
# SAST fica em codeql.yml; DAST e scan de imagem ficam em cd.yml.
name: Security

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  schedule:
    # Toda segunda-feira às 06:00 UTC: pega CVEs publicadas sem que o código tenha
    mudado.
    - cron: '0 6 * * 1'
  workflow_dispatch:

permissions:
  contents: read
  security-events: write

jobs:
  secrets:
    name: Secrets scanning (bloqueante)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          # Histórico completo: um segredo removido no último commit continua no
          histórico.
          fetch-depth: 0

      - name: Gitleaks
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

  sca:
    name: SCA · ${ matrix.app }
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        app: [server-api, inner-view-client]
    steps:
      - uses: actions/checkout@v4

      # Bloqueante: apenas CRITICAL/HIGH que já possuem correção publicada.
      - name: Trivy · dependências (bloqueante)
        uses: aquasecurity/trivy-action@0.28.0
        with:
          scan-type: fs
          scan-ref: ${ matrix.app }
          scanners: vuln
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: '1'
          trivy-config: ''

      # Informativo: relatório completo, todas as severidades, sempre publicado.
      - name: Trivy · relatório completo (informativo)
        if: always()
        uses: aquasecurity/trivy-action@0.28.0
        with:
          scan-type: fs
          scan-ref: ${ matrix.app }

```

```

        scanners: vuln
        severity: CRITICAL,HIGH,MEDIUM,LOW
        exit-code: '0'
        format: sarif
        output: trivy-${{ matrix.app }}.sarif

- name: Enviar resultado para o Security tab
  if: always()
  uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: trivy-${{ matrix.app }}.sarif
    category: sca-${{ matrix.app }}

sbom:
  name: SBOM · ${{ matrix.app }}
  runs-on: ubuntu-latest
  strategy:
    matrix:
      app: [server-api, inner-view-client]
  steps:
    - uses: actions/checkout@v4

    # SBOM em CycloneDX, anexado a toda execução: inventário do que foi liberado.
    - name: Gerar SBOM (CycloneDX)
      uses: anchore/sbom-action@v0
      with:
        path: ${{ matrix.app }}
        format: cyclonedx-json
        artifact-name: sbom-${{ matrix.app }}.cdx.json
        output-file: sbom-${{ matrix.app }}.cdx.json

    - uses: actions/upload-artifact@v4
      with:
        name: sbom-${{ matrix.app }}
        path: sbom-${{ matrix.app }}.cdx.json
        retention-days: 90

```

ANEXO C — .GITHUB/WORKFLOWS/CODEQL.YML E .GITHUB/DEPENDABOT.YML

```
# SAST do Gate 2. Analisa o TypeScript da API e do client.
name: CodeQL (SAST)

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
  schedule:
    - cron: '0 7 * * 1'
  workflow_dispatch:

permissions:
  contents: read
  security-events: write
  actions: read

jobs:
  analyze:
    name: Análise estática
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: github/codeql-action/init@v3
        with:
          languages: javascript-typescript
          # security-extended inclui regras de injeção, criptografia fraca e
          # segredos embutidos em código (ex.: fallback de JWT_ACCESS_SECRET).
          queries: security-extended

      - uses: github/codeql-action/analyze@v3
        with:
          category: /language:javascript-typescript
```

```
version: 2
updates:
  - package-ecosystem: npm
    directory: /server-api
    schedule:
      interval: weekly
    open-pull-requests-limit: 5
    labels: [dependencies, security]
  - package-ecosystem: npm
    directory: /inner-view-client
    schedule:
      interval: weekly
    open-pull-requests-limit: 5
    labels: [dependencies, security]
  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: monthly
  - package-ecosystem: docker
    directory: /server-api
    schedule:
      interval: weekly
```

ANEXO D — .GITHUB/WORKFLOWS/CD.YML

Entrega contínua: preview por *pull request*, empacotamento da API, homologação na Vercel, DAST e promoção para produção com aprovação. Cobre os Gates 3 a 6 e não contém passo pendente.

```
# Entrega contínua do Inner View.
#
# O client (Angular/Ionic) é publicado na Vercel; a API é empacotada como imagem
# versionada no GHCR. A integração Git da Vercel está desligada em vercel.json
# (`"github": { "enabled": false }`): quem publica é esta esteira, para que os
# gates de segurança aconteçam ANTES do deploy e não em paralelo a ele.
#
# Gates cobertos aqui: 3 (empacotamento), 4 (homologação), 5 (DAST) e 6 (produção).
#
# O artefato é construído UMA VEZ, no merge em main, e promovido:
#   vercel build --prod → vercel deploy --prebuilt --prod --skip-domain → vercel promote
# O `--skip-domain` cria um deployment de produção que ainda NÃO recebe o domínio.
# Ele é validado em homologação (smoke + DAST) e, na tag, o `promote` apenas aponta
# o domínio para esse mesmo deployment — o que vai ao ar é exatamente o que foi testado.
#
# Secrets necessários no repositório:
#   VERCEL_TOKEN      token de acesso da Vercel
#   VERCEL_ORG_ID     id da conta/time (vercel.json local: .vercel/project.json)
#   VERCEL_PROJECT_ID id do projeto do client na Vercel
# Secret opcional:
#   VERCEL_AUTOMATION_BYPASS_SECRET libera smoke test e DAST quando o projeto usa
#                                   Deployment Protection (Vercel > Settings > Protection)
# Variáveis (Settings > Variables):
#   PRODUCTION_URL    obrigatória para a promoção: domínio de produção do projeto
#   STAGING_ALIAS     opcional: domínio fixo de homologação, para não caçar o link
name: CD

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]      # -> homologação automática
    tags: ['v*']          # -> produção, com aprovação manual
  workflow_dispatch:

concurrency:
  group: cd-${{ github.ref }}
  cancel-in-progress: false

permissions:
  contents: read
  packages: write

env:
  CLIENT_DIR: inner-view-client
  IMAGE: ghcr.io/${{ github.repository }}/server-api
  VERCEL_ORG_ID: ${ secrets.VERCEL_ORG_ID }
  VERCEL_PROJECT_ID: ${ secrets.VERCEL_PROJECT_ID }

jobs:
  # -----
  # Pull request: ambiente efêmero para revisão humana.
  # Não é gate — é o que permite revisar a mudança rodando, e não só o diff.
  # -----
  preview:
    name: Preview do client (pull request)
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    environment:
      name: preview
      url: ${ steps.deploy.outputs.url }
    defaults:
      run:
```

```

    working-directory: inner-view-client
steps:
  - uses: actions/checkout@v4

  - uses: actions/setup-node@v4
    with:
      node-version: 22
      cache: npm
      cache-dependency-path: inner-view-client/package-lock.json

  - name: Instalar a CLI da Vercel
    run: npm install --global vercel@60

  - name: Baixar configuração do environment Preview
    run: vercel pull --yes --environment=preview --token='${{ secrets.VERCEL_TOKEN }}'

  - name: Build
    run: vercel build --token='${{ secrets.VERCEL_TOKEN }}'

  - id: deploy
    name: Publicar preview
    run: |
      url=$(vercel deploy --prebuilt \
        --meta githubCommitSha="$GITHUB_SHA" \
        --meta githubPrNumber="${{ github.event.number }}" \
        --token='${{ secrets.VERCEL_TOKEN }}')
      echo "url=$url" >> "$GITHUB_OUTPUT"
      echo "### Preview: $url" >> "$GITHUB_STEP_SUMMARY"

# -----
# Gate 3 – empacotamento da API. A imagem só chega ao registry se passar no scan.
# A API ainda não tem host contratado; o que a esteira garante hoje é que o
# artefato existe, é rastreável pelo SHA e tem inventário.
# -----
package-api:
  name: API · imagem + scan + SBOM (Gate 3)
  if: github.event_name != 'pull_request'
  runs-on: ubuntu-latest
  outputs:
    tag: ${ steps.meta.outputs.tag }
  steps:
    - uses: actions/checkout@v4

    - id: meta
      run: echo "tag=${GITHUB_SHA::12}" >> "$GITHUB_OUTPUT"

    - uses: docker/setup-buildx-action@v3

    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }

    - name: Build
      uses: docker/build-push-action@v6
      with:
        context: ./server-api
        load: true
        tags: ${ env.IMAGE }:${ steps.meta.outputs.tag }

# Bloqueante: vulnerabilidade crítica/alta com correção disponível não vai para o registry.
- name: Trivy · imagem (bloqueante)
  uses: aquasecurity/trivy-action@0.28.0
  with:
    image-ref: ${ env.IMAGE }:${ steps.meta.outputs.tag }
    severity: CRITICAL,HIGH
    ignore-unfixed: true
    exit-code: '1'

- name: SBOM da imagem
  uses: anchore/sbom-action@v0
  with:
    image: ${ env.IMAGE }:${ steps.meta.outputs.tag }

```

```

    format: cyclonedx-json
    output-file: sbom-image.cdx.json

- uses: actions/upload-artifact@v4
  with:
    name: sbom-image
    path: sbom-image.cdx.json
    retention-days: 90

- name: Push
  uses: docker/build-push-action@v6
  with:
    context: ./server-api
    push: true
    tags: |
      ${ env.IMAGE }:${ steps.meta.outputs.tag }
      ${ env.IMAGE }:${ github.ref_type == 'tag' && github.ref_name || 'edge' }}

# -----
# Gates 3 e 4 - client: build único, deployment imutável, homologação e smoke.
# -----
staging:
  name: Client · build, SBOM e homologação (Gates 3 e 4)
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment:
    name: staging
    url: ${ steps.deploy.outputs.url }
  outputs:
    url: ${ steps.deploy.outputs.url }
  defaults:
    run:
      working-directory: inner-view-client
  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-node@v4
      with:
        node-version: 22
        cache: npm
        cache-dependency-path: inner-view-client/package-lock.json

    - name: Instalar a CLI da Vercel
      run: npm install --global vercel@60

    - name: Baixar configuração do environment Production
      run: vercel pull --yes --environment=production --token='${ secrets.VERCEL_TOKEN }'

    # Build de produção. É este bundle, e nenhum outro, que será promovido na tag.
    - name: Build de produção
      run: vercel build --prod --token='${ secrets.VERCEL_TOKEN }'

    # SBOM do que foi efetivamente liberado, indexado pelo SHA do commit.
    - name: SBOM do client (CycloneDX)
      uses: anchore/sbom-action@v0
      with:
        path: inner-view-client
        format: cyclonedx-json
        output-file: sbom-client.cdx.json
        artifact-name: sbom-client-${ github.sha }.cdx.json

    # --skip-domain: deployment de produção que ainda não recebe o domínio.
    - id: deploy
      name: Publicar deployment (sem apontar o domínio)
      run: |
        url=$(vercel deploy --prebuilt --prod --skip-domain \
          --meta githubCommitSha="$GITHUB_SHA" \
          --token='${ secrets.VERCEL_TOKEN }')
        echo "url=$url" >> "$GITHUB_OUTPUT"
        echo "### Homologação: $url" >> "$GITHUB_STEP_SUMMARY"

    # Opcional: URL fixa de homologação, para o grupo não precisar caçar o link.
    - name: Apontar o alias de homologação
      if: vars.STAGING_ALIAS != ''

```

```

        run: vercel alias set '${{ steps.deploy.outputs.url }}' '${{ vars.STAGING_ALIAS }}' --
token='${{ secrets.VERCEL_TOKEN }}'

# Bloqueante: se o deployment não responde 200 com a aplicação dentro,
# não existe o que promover.
- name: Smoke test
  env:
    BYPASS: ${{ secrets.VERCEL_AUTOMATION_BYPASS_SECRET }}
  run: |
    url='${{ steps.deploy.outputs.url }}'
    [ -n "$BYPASS" ] && url="$url?x-vercel-protection-bypass=$BYPASS&x-vercel-set-bypass-
cookie=true"
    curl -fsS --retry 5 --retry-delay 5 --retry-all-errors "$url" | grep -q '<app-root'
    echo "Smoke test OK"

# -----
# Gate 5 – DAST sobre a aplicação publicada em homologação.
# Alerta High reprova; Medium e Low viram aviso (.zap/rules.tsv).
# -----
dast-staging:
  name: DAST em homologação (Gate 5)
  needs: staging
  runs-on: ubuntu-latest
  permissions:
    contents: read
    issues: write
  steps:
    - uses: actions/checkout@v4

    - id: target
      env:
        BYPASS: ${{ secrets.VERCEL_AUTOMATION_BYPASS_SECRET }}
      run: |
        url='${{ needs.staging.outputs.url }}'
        # A Deployment Protection da Vercel bloquearia o ZAP. O parâmetro de bypass
        # devolve um cookie que o ZAP mantém durante a varredura.
        [ -n "$BYPASS" ] && url="$url?x-vercel-protection-bypass=$BYPASS&x-vercel-set-bypass-
cookie=true"
        echo "url=$url" >> "$GITHUB_OUTPUT"

    - name: OWASP ZAP Baseline
      uses: zaproxy/action-baseline@v0.12.0
      with:
        target: ${{ steps.target.outputs.url }}
        rules_file_name: .zap/rules.tsv
        cmd_options: '-a -j'
        fail_action: true

# -----
# Gate 6 – produção. Tag v* + aprovação manual no environment `production`.
# Nada é construído aqui: apenas se aponta o domínio para o deployment que já
# passou por smoke e DAST em homologação.
# -----
promote-production:
  name: Promoção para produção (Gate 6)
  if: github.event_name == 'push' && github.ref_type == 'tag'
  runs-on: ubuntu-latest
  # O environment `production` exige revisor aprovador configurado no GitHub:
  # nenhuma release chega ao usuário sem aprovação humana.
  environment:
    name: production
    url: ${{ vars.PRODUCTION_URL }}
  steps:
    - uses: actions/checkout@v4

    # Sem o domínio de produção não há como verificar o que foi promovido.
    # Falhar aqui é melhor do que promover e descobrir depois que o health
    # check testou uma URL vazia.
    - name: Conferir a configuração
      run: |
        if [ -z '${{ vars.PRODUCTION_URL }}' ]; then
          echo "::error::Defina a variável PRODUCTION_URL (Settings > Variables) com o domínio
de produção."
          exit 1

```

```

fi

- name: Instalar a CLI da Vercel
  run: npm install --global vercel@60

# Vincula o diretório ao projeto a partir de VERCEL_ORG_ID/VERCEL_PROJECT_ID,
# para que `promote` e `rollback` saibam sobre qual projeto agir.
- name: Vincular o projeto
  run: vercel pull --yes --environment=production --token='${{ secrets.VERCEL_TOKEN }}'

# Localiza o deployment criado pelo merge em main para ESTE commit.
# Se a tag apontar para um commit que nunca passou por homologação, não há
# o que promover - e a esteira reprova em vez de construir algo novo.
- id: find
  name: Localizar o deployment validado em homologação
  env:
    TOKEN: '${{ secrets.VERCEL_TOKEN }}'
  run: |
    set -euo pipefail
    query="projectId=$VERCEL_PROJECT_ID&meta-
githubCommitSha=$GITHUB_SHA&state=READY&target=production&limit=1"
    case "$VERCEL_ORG_ID" in team_*) query="$query&teamId=$VERCEL_ORG_ID" ;; esac
    resp=$(curl -fsS -H "Authorization: Bearer $TOKEN"
"https://api.vercel.com/v6/deployments?$query")
    url=$(echo "$resp" | jq -r '.deployments[0].url // empty')
    if [ -z "$url" ]; then
      echo "::error::Nenhum deployment pronto para o commit $GITHUB_SHA."
      echo "::error::Marque com tag apenas commits que já passaram por homologação."
      exit 1
    fi
    echo "url=https://$url" >> "$GITHUB_OUTPUT"

# Qual deployment o domínio de produção serve NESTE momento. É para ele que
# o rollback volta, e por isso a consulta é no projeto e não na lista de
# deployments (o mais recente da lista é justamente o que vamos promover).
- name: Guardar a produção atual (para rollback)
  id: current
  env:
    TOKEN: '${{ secrets.VERCEL_TOKEN }}'
  run: |
    set -euo pipefail
    query=""
    case "$VERCEL_ORG_ID" in team_*) query="?teamId=$VERCEL_ORG_ID" ;; esac
    resp=$(curl -fsS -H "Authorization: Bearer $TOKEN" \
      "https://api.vercel.com/v9/projects/$VERCEL_PROJECT_ID$query")
    echo "url=$(echo "$resp" | jq -r '.targets.production.url // empty')" >>
"$GITHUB_OUTPUT"

# Promoção: o domínio de produção passa a apontar para o deployment validado.
- name: Promover '${{ github.ref_name }}'
  run: |
    vercel promote '${{ steps.find.outputs.url }}' --token='${{ secrets.VERCEL_TOKEN }}'
    echo "### Produção: '${{ github.ref_name }}' → '${{ steps.find.outputs.url }}'" >>
"$GITHUB_STEP_SUMMARY"

- name: Health check em produção
  run: |
    curl -fsS --retry 5 --retry-delay 5 --retry-all-errors '${{ vars.PRODUCTION_URL }}' |
grep -q '<app-root'
    echo "Health check OK"

# Rollback: volta o domínio para o deployment anterior, que continua existindo.
# Na Vercel o deployment é imutável, então desfazer é trocar o ponteiro.
- name: Rollback automático em caso de falha
  if: failure() && steps.current.outputs.url != ''
  run: |
    vercel rollback 'https://${{ steps.current.outputs.url }}' --token='${{
secrets.VERCEL_TOKEN }}'
    echo "::warning::Produção revertida para https://${{ steps.current.outputs.url }}"

```

ANEXO E — CONFIGURAÇÃO DE ENTREGA E DO DAST

Arquivos que tornam o deploy executável: a configuração do projeto na Vercel, o script que injeta o endereço da API no build e as regras que definem o que o ZAP reprova.

`inner-view-client/vercel.json` — build, roteamento de SPA e cabeçalhos de segurança. Os cabeçalhos aqui são exatamente os que o ZAP cobra no Gate 5; a integração Git está desligada para que o deploy dependa da esteira.

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "installCommand": "npm ci",
  "buildCommand": "node scripts/set-api-url.mjs && npx ng build --configuration production",
  "outputDirectory": "www",
  "framework": null,
  "github": {
    "enabled": false
  },
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ],
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "Strict-Transport-Security", "value": "max-age=63072000; includeSubDomains; preload" },
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "X-Frame-Options", "value": "DENY" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
        { "key": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=(), payment=()" },
        { "key": "Content-Security-Policy", "value": "default-src 'self'; base-uri 'self'; object-src 'none'; frame-ancestors 'none'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' blob:; font-src 'self' data:; worker-src 'self' blob:; connect-src 'self' https:; form-action 'self'; upgrade-insecure-requests" }
      ]
    },
    {
      "source": "/assets/(.*)",
      "headers": [
        { "key": "Cache-Control", "value": "public, max-age=604800, immutable" }
      ]
    },
    {
      "source": "/index.html",
      "headers": [
        { "key": "Cache-Control", "value": "no-store, max-age=0" }
      ]
    }
  ]
}
```

`inner-view-client/scripts/set-api-url.mjs` — substitui o endereço da API no *bundle* a partir da variável do ambiente. Sem a variável, o build falha.

```

// Injeta a URL da API no bundle de produção a partir da variável de ambiente
// API_URL, definida no projeto da Vercel (uma por environment: Preview e Production).
//
// Motivo: o endereço da API é configuração de ambiente, não código. Antes desta
// esteira o valor estava fixo em environment.prod.ts (`https://api.seudominio.com`),
// um endereço que não existe — o build passava e a aplicação publicada quebrava.
//
// O script falha quando API_URL não está definida, pela mesma regra aplicada ao
// segredo do JWT na API: sem a configuração, a aplicação não sobe em vez de subir
// apontando para lugar nenhum.

import { readFileSync, writeFileSync } from 'node:fs';

const target = new URL('../src/environments/environment.prod.ts', import.meta.url);
const apiUrl = (process.env.API_URL ?? '').trim();

if (!apiUrl) {
  console.error(
    '[set-api-url] API_URL não está definida.\n' +
    '      Configure a variável no projeto da Vercel (Settings > Environment Variables)\n' +
    '      ou exporte API_URL antes de rodar o build.'
  );
  process.exit(1);
}

if (!/^(https:\\\\|\\|\\/).test(apiUrl)) {
  console.error(`[set-api-url] API_URL inválida: "${apiUrl}". Use https:// ou um caminho iniciado por ./`);
  process.exit(1);
}

const source = readFileSync(target, 'utf8');
const patched = source.replace(/apiUrl:\\s*'[^']*'\\/, `apiUrl: '${apiUrl.replace(/\\/+/g, '\\\\')}',`);

if (patched === source) {
  console.error('[set-api-url] Não encontrei a chave apiUrl em environment.prod.ts. Build interrompido.');
```

```

  process.exit(1);
}

writeFileSync(target, patched);
console.log(`[set-api-url] apiUrl = ${apiUrl}`);

```

.zap/rules.tsv — política do DAST: WARN alerta, FAIL reprov a promoção.

```

# Regras do ZAP Baseline sobre o client publicado em homologação na Vercel.
# WARN = alerta, não bloqueia. FAIL = bloqueia a promoção para produção.
# Os cabeçalhos marcados como FAIL são os que inner-view-client/vercel.json
# configura: o gate existe para detectar quando alguém os remover.
# id      nível  descrição
10015    WARN    (Cabeçalho Cache-Control ausente)
10027    WARN    (Comentários informativos no código)
10096    WARN    (Timestamp divulgado)
10063    WARN    (Permissions-Policy ausente ou incompleta)
90004    WARN    (Isolamento de site insuficiente contra Spectre)
10035    FAIL    (HSTS ausente em endpoint HTTPS)
10020    FAIL    (Anti-clickjacking ausente)
10021    FAIL    (X-Content-Type-Options ausente)
10038    FAIL    (Content-Security-Policy ausente)

```

ANEXO F — EVIDÊNCIAS DO DIAGNÓSTICO

Saídas obtidas na raiz do repositório em 26/09/2026, reproduzidas com os comandos indicados.

```
$ git rev-list --count HEAD ; git rev-list --count --merges HEAD ; git branch -a ; git tag | wc -l
19
0
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
0
$ git shortlog -sn HEAD
   11  Fabricio Varisco
    6  MatheusThiesen
    2  fabvarisco

$ ls .github          # antes desta atividade
ls: .github: No such file or directory

$ cd server-api && yarn test
PASS src/modules/virtual-tours/services/create-virtual-tour.service.spec.ts
... (10 suites)
Test Suites: 10 passed, 10 total
Tests:       52 passed, 52 total
Time:        5.41 s

$ yarn test:cov
File                                | % Stmts | % Branch | % Funcs | % Lines
All files                            |    28   |   19.14  |   32.28  |   27.75

$ npx eslint "src/**/*.ts"          # sem --fix, como a esteira roda
10:17 error   Replace ... with ...   prettier/prettier
x 89 problems (89 errors, 0 warnings)
   88 errors and 0 warnings potentially fixable with the `--fix` option.

$ yarn audit                          # API - 827 dependencias
vulnerabilities: { info: 0, low: 6, moderate: 35, high: 136, critical: 0 }

$ cd ../inner-view-client && npm audit
vulnerabilities: { info: 0, low: 3, moderate: 16, high: 35, critical: 1, total: 55 }
dependencies:    { prod: 37, dev: 1564, optional: 222, total: 1608 }

$ npx ng lint
src/app/services/language.service.ts
  9:15 error   Prefer using the inject() function over constructor parameter injection
x 1 problem (1 error, 0 warnings)

$ npx ng test --watch=false --browsers=ChromeHeadless
Chrome Headless HomePage should create FAILED
  NotFound: NG0201: No provider found for `HttpClient`.
  Source: Standalone[HomePage]. Path: PropertyService -> HttpClient
Executed 5 of 5 (3 FAILED)
TOTAL: 3 FAILED, 2 SUCCESS

$ yarn test:e2e                      # API
Can't find a root directory while resolving a config file path.
Provided path to resolve: ./test/jest-e2e.json

$ grep -rn "JWT_ACCESS_SECRET" server-api/src
src/common/strategies/jwt-access.strategy.ts:18:                                secretOrKey:
process.env.JWT_ACCESS_SECRET ?? 'access-secret',
src/modules/auth/services/token.service.ts:20:                                secret:
process.env.JWT_ACCESS_SECRET ?? 'access-secret',

$ git ls-files | grep -i env
inner-view-client/src/environments/environment.prod.ts
inner-view-client/src/environments/environment.ts
server-api/.env.example          # nenhum .env real versionado
```

```
$ grep -c "USER" server-api/Dockerfile # imagem executa como root
0
```